Comparación experimental de Insertion Sort y QuickSort en arreglos de números

Experimental comparison of Insertion Sort and QuickSort on numeric arrays

Marco Antonio Phatti Santos 2026-119011 · Universidad Nacional Jorge Basadre Grohmann
Escuela Profesional de Ingeniería en Informática y Sistemas · Tacna, Perú
Correo institucional: mphattis@unjbg.edu.pe

Resumen

Este artículo presenta una comparación experimental de Insertion Sort y QuickSort implementados en C++. Se evaluaron arreglos de 500, 1 000, 2 000 y 5 000 enteros, con cuatro patrones de entrada: aleatorio, ordenado, inverso y con duplicados. Cada combinación se ejecutó siete veces y se informó la mediana de los tiempos medidos con *steady_clock*. Los datos se generaron con una semilla fija y se comprobó el orden final mediante *is_sorted*. En la muestra de 5 000 elementos, Insertion Sort mostró un desempeño especialmente favorable en el arreglo ordenado, mientras QuickSort presentó menor tiempo en los patrones desordenados. Las comparaciones, movimientos e intercambios se contabilizaron según los puntos de instrumentación de cada implementación. Los resultados ilustran la dependencia del rendimiento respecto del patrón de datos y del tamaño de entrada; no deben extrapolarse sin considerar el entorno de ejecución.

Palabras clave: Insertion Sort, QuickSort, ordenamiento, análisis experimental, C++.

Abstract

This paper reports an experimental comparison of Insertion Sort and QuickSort implemented in C++. Arrays of 500, 1,000, 2,000, and 5,000 integers were evaluated under four input patterns: random, sorted, reverse-sorted, and duplicate-rich. Each combination was executed seven times, and median runtime measured with *steady_clock* was reported. Data were generated with a fixed seed, and sorted output was verified with *is_sorted*. For the 5,000-element sample, Insertion Sort performed particularly well on already sorted input, whereas QuickSort was faster on the unsorted patterns. Comparisons, movements, and swaps were counted according to the instrumentation points in each implementation. The results illustrate how performance depends on data arrangement and input size and should not be generalized without considering the execution environment.

Keywords: Insertion Sort, QuickSort, sorting, experimental analysis, C++.

1. Introducción

El ordenamiento es una operación fundamental en programación, porque permite organizar datos para facilitar su consulta y procesamiento. La eficiencia de un método no depende únicamente de su diseño: también influyen el número de elementos, la distribución de valores y el estado inicial del arreglo. Por este motivo, una evaluación experimental debe considerar entradas variadas y métricas complementarias.

Insertion Sort recorre la secuencia y coloca cada elemento en su posición dentro de la parte ya ordenada. Es sencillo y puede ser eficiente cuando la entrada está ordenada o casi ordenada; sin embargo, en el caso promedio y en el peor caso su tiempo es cuadrático. QuickSort particiona el arreglo alrededor de un pivote y ordena recursivamente los segmentos. Su rendimiento promedio suele ser O(n log n), aunque una partición desequilibrada puede llevarlo a $O(n^2)$.

El objetivo de este trabajo es comparar las implementaciones proporcionadas mediante tiempo de ejecución, comparaciones y operaciones de movimiento/intercambio. Se busca observar cómo cambian los resultados en cuatro patrones de entrada y en varios tamaños.

2. Método

2.1. Entorno y configuración

Se compiló una implementación de prueba en C++17 con g++ 14.2.0 y la opción de optimización -O2. La ejecución se realizó en un entorno Linux x86_64. Se usó una semilla fija (20261029) para facilitar la repetición de la muestra. La máquina concreta de destino puede producir tiempos distintos; el entorno indicado describe la ejecución de referencia del documento.

2.2. Patrones y tamaños de entrada

Se analizaron cuatro tamaños: n = 500, 1 000, 2 000 y 5 000. Para cada tamaño se construyeron cuatro patrones: (1) aleatorio, con enteros entre 0 y 9 999; (2) ordenado, mediante ordenamiento ascendente de la muestra aleatoria; (3) inverso, invirtiendo la secuencia ordenada; y (4) duplicados, con enteros entre 0 y 9. Cada algoritmo recibió una copia de la misma entrada. Se realizaron siete repeticiones por combinación y se calculó la mediana del tiempo.

2.3. Métricas y validación

El tiempo se midió con *steady_clock* y se expresó en milisegundos. Insertion Sort contabilizó comparaciones entre la clave y los elementos anteriores, así como escrituras por desplazamiento o colocación. QuickSort contabilizó comparaciones dentro de los recorridos de los índices e intercambios entre posiciones distintas. Después de cada ejecución se verificó que el arreglo estuviera ordenado con *is_sorted*. Los contadores son específicos de esta instrumentación y no representan necesariamente operaciones de máquina equivalentes entre ambos algoritmos.

3. Resultados

3.1. Comparación de tiempo y operaciones para n = 5 000

La Tabla 1 presenta las medianas de tiempo y las comparaciones contabilizadas para la entrada de mayor tamaño. QuickSort registró menor tiempo en los patrones aleatorio, inverso y con duplicados; Insertion Sort fue más rápido con la entrada ya ordenada.

Tabla 1. Tiempo mediano y comparaciones contabilizadas (n = 5 000).

Patrón	Insertion (ms)	QuickSort (ms)	Comp. Insertion	Comp. QuickSort
Aleatorio	5.006833	0.434090	6 244 025	91 178
Ordenado	0.013870	0.078973	4 999	65 933
Inverso	10.000234	0.088729	12 497 314	65 932
Duplicados	4.367049	0.170200	5 655 363	67 191

Nota. Los contadores corresponden a la primera repetición; los tiempos corresponden a la mediana de siete ejecuciones.

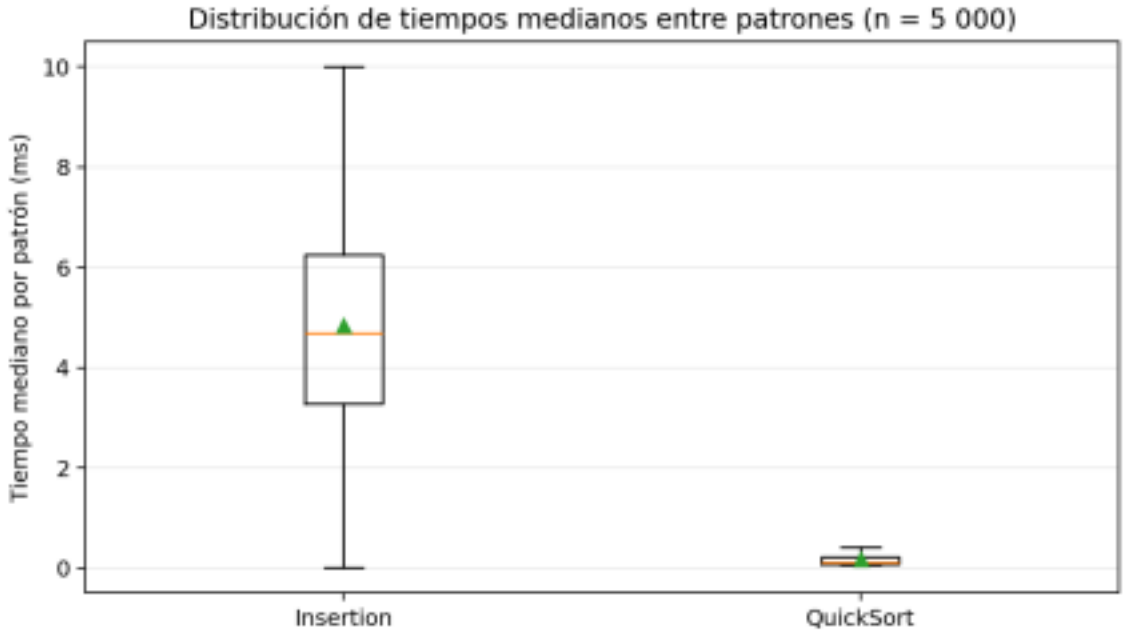


Figura 1. Distribución de tiempos medianos entre los cuatro patrones de entrada para n = 5 000.

3.2. Escalamiento por tamaño

La Figura 2 resume el tiempo mediano promedio entre patrones para cada tamaño. El crecimiento de Insertion Sort se vuelve más marcado al aumentar n, especialmente porque los casos aleatorio e inverso requieren numerosas comparaciones y desplazamientos. QuickSort muestra un crecimiento más moderado en la muestra evaluada.

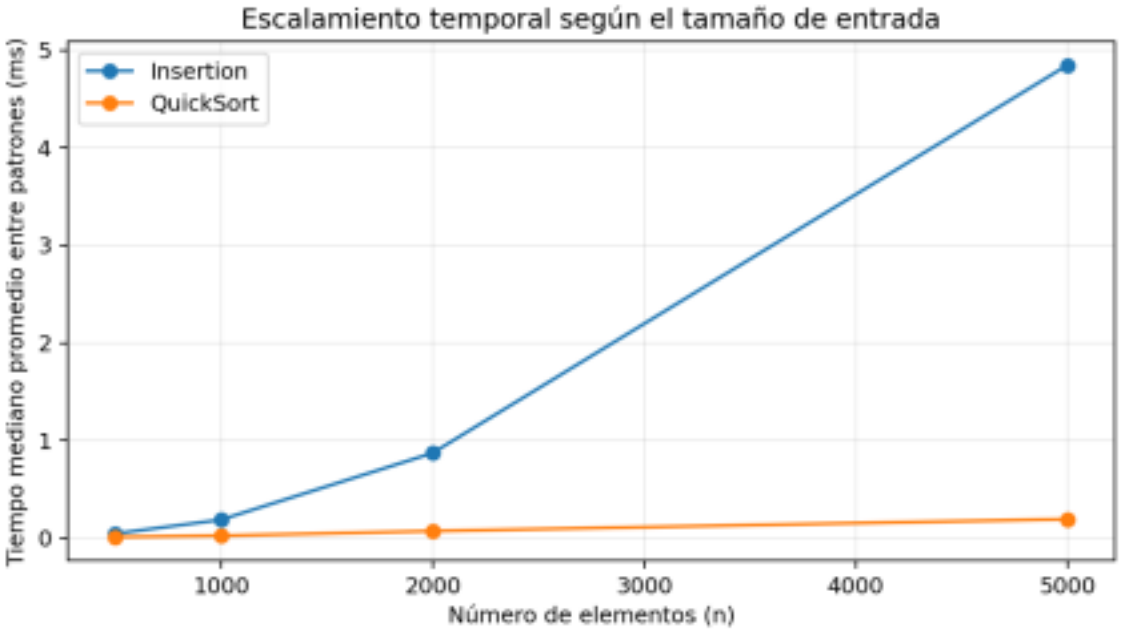


Figura 2. Tiempo mediano promedio entre patrones según el tamaño de entrada.

3.3. Comparaciones y entradas con duplicados

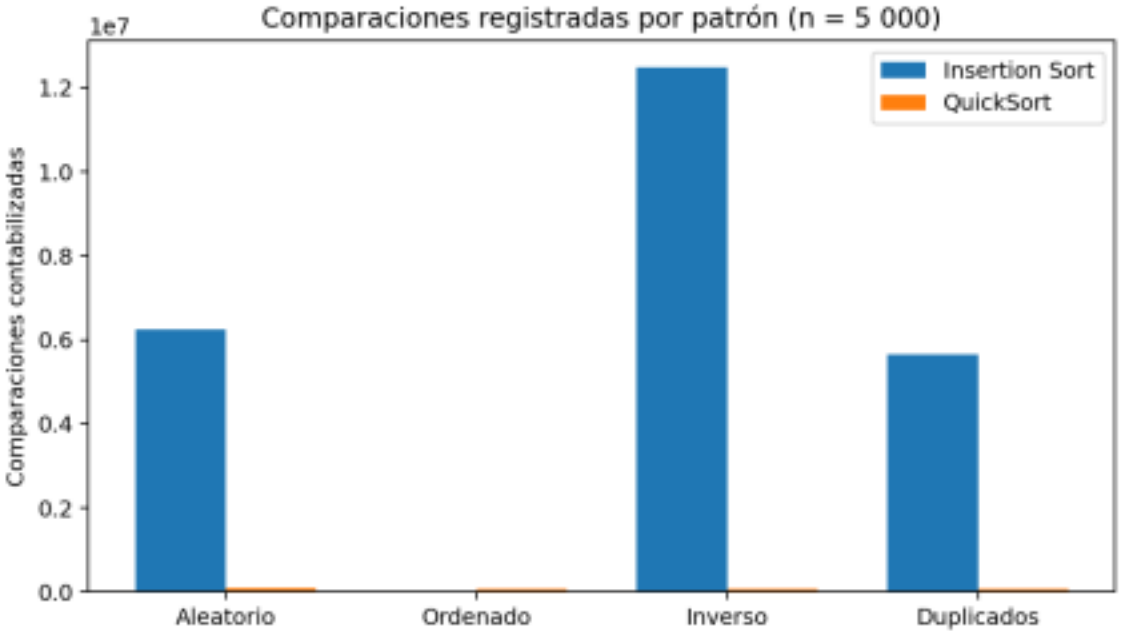


Figura 3. Comparaciones registradas por algoritmo y patrón para n = 5 000.

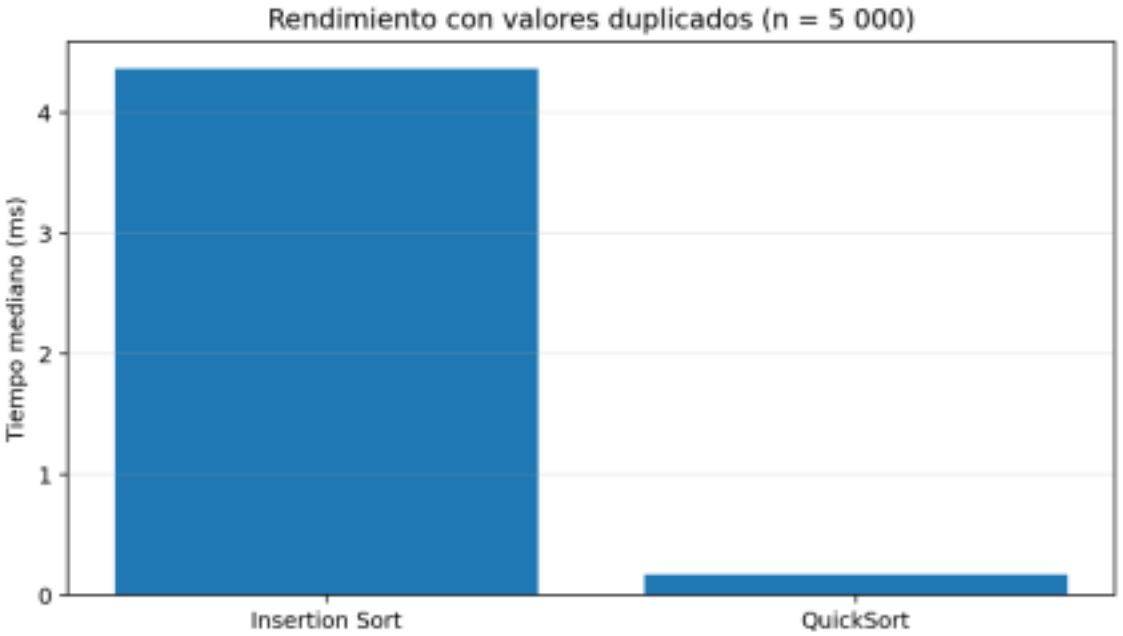


Figura 4. Tiempo mediano con valores duplicados para n = 5 000.

La Tabla 2 reúne movimientos e intercambios contabilizados en la primera repetición. Los contadores de movimientos de Insertion Sort aumentan con fuerza en el arreglo inverso. QuickSort puede registrar más intercambios en la entrada con duplicados que en otros patrones, aunque eso no implica por sí solo un mayor tiempo total.

Tabla 2. Movimientos e intercambios contabilizados (n = 5 000).

Patrón	**Movimientos Insertion**	**Intercambios QuickSort**
Aleatorio	6 244 036	14 700
Ordenado	4 999	1 067
Inverso	12 501 266	3 567
Duplicados	5 655 363	25 486

4. Discusión

Los resultados experimentales coinciden con el comportamiento teórico general de los algoritmos. Insertion Sort es una opción razonable para entradas pequeñas o ya ordenadas, pero sus operaciones crecen rápidamente cuando debe corregir muchas inversiones. QuickSort resultó más rápido en los patrones aleatorio, inverso y con duplicados en esta muestra. La diferencia observada para el arreglo ordenado se explica porque Insertion Sort solo realiza una comparación fallida por elemento, mientras que QuickSort continúa realizando particiones y llamadas recursivas.

La presencia de duplicados merece atención. El particionado implementado avanza mientras los valores sean menores o mayores que el pivote, y realiza intercambios cuando los índices no coinciden. Esta estrategia termina correctamente en los datos ensayados, pero el número de intercambios puede variar con la distribución y la elección del pivote. No debe concluirse que QuickSort sea siempre estable: la implementación no conserva necesariamente el orden relativo de elementos iguales.

La validez de los resultados se limita a la configuración evaluada. El compilador, las opciones de optimización, la arquitectura, la carga del sistema y la semilla afectan las mediciones. Además, los contadores no son totalmente equivalentes: un movimiento de Insertion Sort no tiene el mismo significado operacional que un intercambio de QuickSort. La comparación debe centrarse en las tendencias y acompañarse de los tiempos.

5. Conclusiones

En las pruebas con 5 000 elementos, QuickSort obtuvo menores tiempos medianos para arreglos aleatorios, inversos y con duplicados, mientras Insertion Sort fue superior para datos previamente ordenados. Esto confirma que la selección de un algoritmo debe considerar el patrón de entrada y no únicamente el tamaño del arreglo.

El análisis de operaciones mostró un costo elevado de Insertion Sort en entradas inversas y una variación de intercambios de QuickSort según el patrón, especialmente con duplicados. El uso de siete repeticiones y la mediana aporta una estimación menos sensible a valores extremos, aunque no elimina la variabilidad del entorno.

Como continuación, se recomienda repetir el experimento en el equipo del curso, anotar sus especificaciones, conservar la semilla y ampliar los tamaños. También conviene incorporar una variante de QuickSort de tres vías para evaluar específicamente arreglos con muchos valores repetidos.

Referencias

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to algorithms* (4th ed.). The MIT Press.

Sedgewick, R., & Wayne, K. (2011). *Algorithms* (4th ed.). Addison-Wesley Professional. https://algs4.cs.princeton.edu/home/